A BEGINNER-FRIENDLY GUIDE

Full Stack in Advanced PHP with AI

Frontend | PHP | MySQL | Laravel | Integration | Security

Course Overview - Basic Explanation of 6 Core Modules

Introduction

What is Full Stack Development?

A **full stack developer** builds both the part of a website users see (the **frontend**) and the part that runs on the server (the **backend**), including the database that stores information. This guide explains, in simple terms, the six modules needed to become a full stack developer using **PHP**, one of the most widely used server-side languages on the web.

How the pieces fit together

Frontend	HTML, CSS and JavaScript create the pages, layout and interactivity in the browser.
Backend	PHP and Laravel receive requests, apply logic, and send back responses.
Database	MySQL stores users, products, posts and any other data permanently.
Security & Deploy	Protects the app from attacks and puts it live on a server.
AI	AI tools help with generating code, debugging, writing tests and adding smart features such as chatbots through APIs.

Learning roadmap

1	**Frontend Foundations**
2	**Core PHP and OOP**
3	**MySQL and Database Design**
4	**Laravel Framework**
5	**Frontend Integration**
6	**Security, Git and Deployment**

1 Frontend Foundations

HTML5 | CSS3 | Responsive layouts | JavaScript ES6

The frontend is everything the visitor sees and clicks. Mastering these basics is the first step because every backend ultimately produces HTML, CSS and JavaScript for the browser.

Key topics explained

HTML5	Defines page structure using elements like headings, forms, images and semantic tags (header, nav, main, footer).
CSS3	Styles the page: colors, fonts, spacing, Flexbox and Grid layouts, transitions and animations.
Responsive layouts	Makes a site look good on phones, tablets and desktops using media queries and flexible units.
JavaScript ES6	Adds behavior. Modern features: let/const, arrow functions, template strings, modules, promises.

Why it matters

- Good structure improves accessibility and SEO.
- Responsive design is essential as most users browse on mobile.
- JavaScript is required later for AJAX, Vue and dynamic pages.

Quick example

```
<div class="card">Hello</div>

.card { display: flex; padding: 1rem; }
@media (max-width: 600px) { .card { flex-direction: column; } }

const greet = (name) => `Hello, ${name}!`;
```

2 Core PHP and OOP

Syntax | Functions | Classes | Namespaces | Sessions

PHP is a server-side language: code runs on the server and the browser receives only the result. Learning core PHP and Object-Oriented Programming (OOP) gives you the foundation for frameworks like Laravel.

Key topics explained

Syntax	Variables start with $, statements end with ;, and code lives inside <?php ... ?> tags. Includes loops, conditions and arrays.
Functions	Reusable blocks of code with parameters, return values and type hints.
Classes and OOP	Objects bundle data and behavior. Core ideas: encapsulation, inheritance, polymorphism, interfaces.
Namespaces	Organise classes into groups and avoid name conflicts; used with autoloading.
Sessions	Store user data (such as login status) across multiple page requests.

Why it matters

- OOP keeps large projects organised and reusable.
- Laravel is built entirely on OOP concepts.
- Sessions power logins and shopping carts.

Quick example

```
<?php
namespace App;

class User {
    public function __construct(private string $name) {}
    public function hello(): string { return "Hi, {$this->name}"; }
}

session_start();
$_SESSION['user'] = 'Asha';
```

3 MySQL and Database Design

Queries | Joins | Normalisation | PDO

Almost every real application stores data. MySQL is a popular relational database that keeps information in tables made of rows and columns.

Key topics explained

Queries	SQL commands: SELECT to read, INSERT to add, UPDATE to change, DELETE to remove data.
Joins	Combine rows from related tables, e.g. orders with their customers (INNER JOIN, LEFT JOIN).
Normalisation	Organising tables to remove duplicate data and keep it consistent (1NF, 2NF, 3NF).
PDO	PHP Data Objects: a safe, consistent way for PHP to talk to databases using prepared statements.

Why it matters

- Good design makes apps faster and easier to maintain.
- Primary and foreign keys link tables together.
- PDO prepared statements help prevent SQL injection.

Quick example

```
SELECT users.name, orders.total
FROM users
INNER JOIN orders ON orders.user_id = users.id;

$pdo = new PDO("mysql:host=localhost;dbname=shop", $user, $pass);
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
$stmt->execute([$id]);
```

4 Laravel Framework

Routing | Blade | Eloquent | Authentication | REST APIs

Laravel is the most popular PHP framework. It provides ready-made tools so you can build secure, well-structured applications faster, following the MVC (Model-View-Controller) pattern.

Key topics explained

Routing	Maps URLs to controllers or functions, e.g. /posts shows a list of posts.
Blade	Laravel's templating engine for writing clean HTML with variables, loops and layouts.
Eloquent ORM	Work with database tables as PHP objects instead of writing raw SQL.
Authentication	Built-in registration, login, logout and password reset (Breeze, Jetstream, Sanctum).
REST APIs	Return JSON data for mobile apps and JavaScript frontends using API routes and resources.

Why it matters

- Saves development time with conventions and built-in features.
- Includes migrations, validation, queues and testing.
- Large community and excellent documentation.

Quick example

```
// routes/web.php
Route::get('/posts', [PostController::class, 'index']);

// Controller
public function index() {
    return view('posts.index', ['posts' => Post::latest()->get()]);
}

// Blade
@foreach ($posts as $post) <h2>{{ $post->title }}</h2> @endforeach
```

5 Frontend Integration

Fetch API | AJAX | Bootstrap or Tailwind | Vue basics

This module connects the frontend with the backend so pages can update without a full reload, and makes the interface look polished using modern UI tools.

Key topics explained

Fetch API	Built-in JavaScript function to request data from a server and handle the response with promises.
AJAX	Technique of exchanging data with the server in the background, giving a smooth experience.
Bootstrap / Tailwind	Bootstrap offers ready-made components; Tailwind uses small utility classes for custom designs.
Vue basics	A progressive JavaScript framework: reactive data, components, and directives like v-for and v-if. Works well with Laravel.

Why it matters

- Creates fast, app-like experiences (live search, forms without reload).
- Frameworks speed up consistent, responsive UI design.
- Prepares you for building Single Page Applications.

Quick example

```
fetch('/api/posts')
  .then(res => res.json())
  .then(data => console.log(data));

<div id="app">
  <li v-for="p in posts">{{ p.title }}</li>
</div>
```

6 Security, Git and Deployment

SQL injection | CSRF | Git | Composer | Hosting

Building the app is only half the job. Developers must protect it from attackers, track changes in code, manage packages, and finally publish it for the world to use.

Key topics explained

SQL injection	Attack where malicious SQL is inserted via input. Prevent with prepared statements and validation.
CSRF	Cross-Site Request Forgery tricks a logged-in user into unwanted actions. Prevent with CSRF tokens (Laravel uses @csrf).
Git	Version control to record changes, work in branches and collaborate using GitHub or GitLab.
Composer	PHP dependency manager used to install libraries and Laravel itself.
Hosting	Deploy on shared hosting, a VPS or cloud (DigitalOcean, AWS) with a web server, PHP, database and HTTPS.

Why it matters

- Also remember XSS protection, password hashing and input validation.
- Git history lets you undo mistakes safely.
- Keep secrets in .env files and never commit them.

Quick example

```
git init
git add .
git commit -m "First commit"
git push origin main

composer create-project laravel/laravel myapp
<form method="POST"> @csrf ... </form>
```

Summary

Putting it all together

By completing these six modules you will be able to design, build, secure and publish a complete web application. Use AI assistants throughout the journey to explain errors, generate boilerplate, review code and suggest improvements, but always understand and test what you ship.

1. Frontend	Build the interface the user sees.
2. Core PHP and OOP	Write structured server-side logic.
3. MySQL	Store and retrieve data safely.
4. Laravel	Build full applications and APIs quickly.
5. Integration	Connect the UI and backend smoothly.
6. Security and Deploy	Protect, version and launch your project.

Suggested practice projects

- Personal portfolio website (Module 1)
- Login and registration system in core PHP (Modules 2-3)
- Blog or task manager in Laravel with authentication (Module 4)
- Live search and comments using Fetch API and Vue (Module 5)
- Deploy your project online with Git and HTTPS (Module 6)